



Divizibilitate

Ciurul lui Eratostene · Descompunere în factori primi · Suma și numărul divizorilor

Andrei Mihai Ciobanu

Algolymp Summer School

Divizibilitate

Numărul și suma divizorilor

Numere prime și verificarea primalității

Descompunerea în factori primi

Ciurul lui Eratostene

Recapitulare

Divizibilitate

Definiție

Spunem că a **îl divide** pe b (scriem $a \mid b$) dacă există un număr întreg c astfel încât $b = a \cdot c$.

- a se numește **divizor** al lui b
- b se numește **multiplu** al lui a

Exemple:

- $3 \mid 12$ deoarece $12 = 3 \cdot 4$
- $5 \mid 30$ deoarece $30 = 5 \cdot 6$
- $4 \nmid 10$ deoarece nu există c întreg cu $10 = 4 \cdot c$

Divizori proprii și improprii

Definiție

Divizorii proprii ai lui n sunt toți divizorii săi **diferiți de 1 și de n** .

Divizorii improprii ai lui n sunt 1 și n însuși.

Exemplu: $n = 12$, divizorii sunt 1, 2, 3, 4, 6, 12

- Proprii: 2, 3, 4, 6
- Improprii: 1, 12

Găsirea divizorilor — varianta $O(n)$

Idee

Verificăm fiecare $d \in \{1, 2, \dots, n\}$ dacă $d \mid n$.

Exemplu: $n = 12 \Rightarrow$ divizori găsiți: 1, 2, 3, 4, 6, 12

- Corect, dar ineficient pentru n mare ($n > 10^7$)
- Putem face mai bine?

▷ $O(f(n))$ = notăție pentru complexitate; descrie câte operații face algoritmul raportat la n

Găsirea divizorilor — reducere la $\lfloor n/2 \rfloor$ iterații

Exemplu: $n = 12$, divizorii sunt 1, 2, 3, 4, 6, 12. Cel mai mare divizor propriu este $6 = 12/2$. De ce nu poate fi mai mare?

Demonstrație

Dacă $d \mid n$, atunci $n = d \cdot k$ cu $k \in \mathbb{N}^*$.

Dacă $d \neq n$, atunci $k \geq 2$, deci $d = \frac{n}{k} \leq \frac{n}{2}$.

- Căutăm în $\{1, 2, \dots, \lfloor \frac{n}{2} \rfloor\}$ și adăugăm n separat
- De 2 ori mai rapid, dar încă inefficient pentru n mare

Observație cheie

Dacă $d \mid n$, atunci și $\frac{n}{d} \mid n$.

Divizorii vin în **perechi** $(d, n/d)$, cu $d \leq \sqrt{n} \leq n/d$.

- Căutăm doar $d \in \{1, 2, \dots, \lfloor \sqrt{n} \rfloor\}$
- Pentru fiecare d găsit, adăugăm și perechea n/d
- **Atenție:** dacă $d = \sqrt{n}$ (pătrat perfect), adăugăm d o singură dată

Exemplu: $n = 36$, $\sqrt{36} = 6$

$d = 1 \rightarrow (1, 36)$, $d = 2 \rightarrow (2, 18)$, $d = 3 \rightarrow (3, 12)$, $d = 4 \rightarrow (4, 9)$,
 $d = 6 \rightarrow (6)$

Găsirea divizorilor — implementare $O(\sqrt{n})$

```
1 void find_divisors(int n) {  
2     for (int d = 1; d * d <= n; d++) {  
3         if (n % d == 0) {  
4             cout << d << " ";  
5             if (d != n / d) {  
6                 cout << n / d << " ";  
7             }  
8         }  
9     }  
10 }
```

- Divizorii nu sunt găsiți în ordine crescătoare
- **Overflow**: $d * d$ poate depăși INT_MAX ($2^{31} - 1$) când n se apropie de $\text{INT_MAX} \rightarrow$ fix rapid: `(long long)d * d <= n`
- Timp: $O(\sqrt{n})$, spațiu: $O(1)$

▷ `void find_divisors(int n)` este o **funcție C++** care primește n și afișează divizorii

Numărul și suma divizorilor

Numărul și suma divizorilor în $O(\sqrt{n})$

Folosind aceeași abordare, calculăm numărul divizorilor ($d(n)$ sau $\tau(n)$) și suma divizorilor ($\sigma(n)$):

- Pentru fiecare pereche $(d, n/d)$ cu $d < n/d$: adăugăm **2** la τ și $d + n/d$ la σ
- Dacă $d = n/d$ (pătrat perfect): adăugăm **1** la τ și d la σ

Observație — pătrate perfecte

n este pătrat perfect $\Leftrightarrow \tau(n)$ este **impar**.

Toate celelalte numere au număr par de divizori.

Numărul și suma divizorilor — implementare

```
1 pair<int, long long> nrdiv_sumdiv(int n) {
2     int nrdiv = 0;
3     long long sumdiv = 0;
4     for (int d = 1; d * d <= n; d++) {
5         if (n % d == 0) {
6             if (d == n / d) {
7                 nrdiv += 1;
8                 sumdiv += d;
9             } else {
10                 nrdiv += 2;
11                 sumdiv += d + n / d;
12             }
13         }
14     }
15     return { nrdiv, sumdiv };
16 }
```

Exemplu: $n = 12 \Rightarrow \tau(n) = 6, \sigma(n) = 28$

▷ `pair<int, long long>` = tip STL care stochează 2 valori; `.first` și `.second` le accesează

Numere prime și verificarea primalității

Numere prime și compuse

Număr prim

Un număr natural $p \geq 2$ este **prim** dacă are exact **2** divizori: 1 și p .

Număr compus

Un număr natural $n \geq 2$ este **compus** dacă are **mai mult de 2** divizori.

- Prime: 2, 3, 5, 7, 11, 13, 17, 19, ...
- Compuse: 4, 6, 8, 9, 10, 12, ...
- 0 și 1 nu sunt **nici prime, nici compuse**

Verificarea primalității

Idee

n este prim $\Leftrightarrow$ nu are niciun divizor d cu $1 < d < n$.

Aplicăm din nou optimizarea cu $\sqrt{n}$:

- Dacă n are măcar un divizor propriu, atunci are măcar un divizor propriu $\leq \sqrt{n}$
- Căutăm în $\{2, 3, \dots, \lfloor \sqrt{n} \rfloor\}$
- Dacă nu găsim niciunul, n este prim

Cazuri speciale

$n < 2 \rightarrow$ nici prim, nici compus | $n = 2 \rightarrow$ prim

Verificarea primalității — implementare

```
1 bool is_prime(int n) {  
2     if (n < 2) {  
3         return false;  
4     }  
5     for (int d = 2; d * d <= n; d++) {  
6         if (n % d == 0) {  
7             return false;  
8         }  
9     }  
10    return true;  
11 }
```

- Timp: $O(\sqrt{n})$, spațiu: $O(1)$
- Eficient pentru un singur număr; inefficient dacă testăm toate numerele până la n

Descompunerea în factori primi

Observație cheie

Cel mai mic divizor al lui n mai mare ca 1 este **întotdeauna prim**.

De ce? Fie p cel mai mic divizor > 1 al lui n . Dacă p ar fi compus, ar avea un divizor q cu $1 < q < p$. Dar atunci $q \mid p$ și $p \mid n$, deci $q \mid n$ — contradicție, deoarece p nu ar mai fi cel mai mic divizor al lui n .

Teorema fundamentală a aritmeticii

Orice $n \geq 2$ se scrie **unic** ca produs de puteri de prime:

$$n = p_1^{a_1} \cdot p_2^{a_2} \cdots p_k^{a_k}, \quad p_1 < p_2 < \cdots < p_k$$

Idee

Împărțim repetat n la cel mai mic factor $p \leq \sqrt{n}$.

La final, dacă $n > 1$, restul n este un factor prim cu exponentul 1.

Exemplu: $n = 360$

- $360 \div 2 = 180 \div 2 = 90 \div 2 = 45 \Rightarrow 2^3$
- $45 \div 3 = 15 \div 3 = 5 \Rightarrow 3^2$
- $5 > 1 \Rightarrow 5^1$
- Rezultat: $360 = 2^3 \cdot 3^2 \cdot 5$

Descompunere în factori primi — implementare

```
1 vector<pair<int, int>> factorize(int n) {
2     vector<pair<int, int>> factors; // (factor prim, exponent)
3     for (int p = 2; p * p <= n; p++) {
4         if (n % p == 0) {
5             int e = 0;
6             while (n % p == 0) {
7                 e++;
8                 n /= p;
9             }
10            factors.push_back({ p, e });
11        }
12    }
13    if (n > 1) {
14        factors.push_back({ n, 1 });
15    }
16    return factors;
17 }
```

Tim: $O(\sqrt{n})$, spațiu: $O(\log n)$ — cel mult $\log_2 n$ factori primi distincți

▷ $\log_2 n$ = de câte ori împarți n la 2 până ajungi la 1; $\log_2(10^6) \approx 20$. *vector* = listă dinamică (STL)

De la descompunere la $\tau(n)$ și $\sigma(n)$

Fie $n = p_1^{a_1} \cdot p_2^{a_2} \cdots p_k^{a_k}$. Orice divizor al lui n are forma $p_1^{b_1} \cdot p_2^{b_2} \cdots p_k^{b_k}$ cu $0 \leq b_i \leq a_i$.

Numărul de divizori

Fiecare b_i poate lua $a_i + 1$ valori, deci:

$$\tau(n) = (a_1 + 1)(a_2 + 1) \cdots (a_k + 1)$$

Suma divizorilor

Înmulțind contribuția fiecărui factor prim:

$$\sigma(n) = \prod_{i=1}^k (1 + p_i + p_i^2 + \cdots + p_i^{a_i}) = \prod_{i=1}^k \frac{p_i^{a_i+1} - 1}{p_i - 1}$$

Exemplu: $360 = 2^3 \cdot 3^2 \cdot 5^1$

$$\tau(360) = 4 \cdot 3 \cdot 2 = 24 \quad \sigma(360) = (1+2+4+8)(1+3+9)(1+5) = 1170$$

Ciurul lui Eratostene

Motivație — de ce avem nevoie de ciur?

- `is_prime(n)` în $O(\sqrt{n})$ e bun pentru **un singur număr**
- Dacă vrem **toate** primele $\leq n$, aplicând `is_prime` de n ori obținem $O(n\sqrt{n})$
- Putem face mult mai bine: $O(n \log n)$ sau chiar $O(n \log \log n)$

Ciurul lui Eratostene — idee

Principiu

Pornim cu toate numerele ≥ 2 marcate ca **prime**.

Pentru fiecare prim p găsit, marcăm toți multiplii $m \geq 2p$ ai lui p ca **compuși**.

1. Inițializăm `prime[i] = true` pentru $i \geq 2$
2. Pentru $i = 2, 3, \dots, \lfloor \sqrt{n} \rfloor$: dacă `prime[i]`, marcăm $2i, 3i, 4i, \dots$ cu `false`
3. `prime[k] == true` $\Leftrightarrow k$ este prim

Optimizare

Putem porni marcarea de la i^2 în loc de $2i$, deoarece multiplii $i \cdot k$ cu $k < i$ au fost deja marcați la pasul k .

Ciurul lui Eratostene — implementare

```
1  const int nmax = 1000000;
2  bool prime[nmax + 1];
3
4  void sieve() {
5      for (int i = 2; i <= nmax; i++) {
6          prime[i] = true;
7      }
8      for (int i = 2; i * i <= nmax; i++) {
9          if (prime[i]) {
10             for (int j = 2 * i; j <= nmax; j += i) {
11                 prime[j] = false;
12             }
13         }
14     }
15 }
```

Timp: $O(n \log n)$, spațiu: $O(n)$

Extensie — ciur pentru $\tau(n)$ și $\sigma(n)$

Modificăm ciurul pentru a calcula τ și σ pentru **toate** numerele până la n :

```
1 const int nmax = 1000000;
2 int nrdiv[nmax + 1];
3 long long sumdiv[nmax + 1];
4
5 void sieve_divisors() {
6     for (int d = 1; d <= nmax; d++) {
7         for (int m = d; m <= nmax; m += d) {
8             nrdiv[m]++;
9             sumdiv[m] += d;
10        }
11    }
12 }
```

Timp: $O(n \log n)$, spațiu: $O(n)$

Recapitulare

Algoritm	Timp	Spațiu
Găsire divizori	$O(\sqrt{n})$	$O(1)$
$\tau(n)$, $\sigma(n)$ (un număr)	$O(\sqrt{n})$	$O(1)$
Verificare primalitate	$O(\sqrt{n})$	$O(1)$
Descompunere în factori primi	$O(\sqrt{n})$	$O(\log n)$
Ciurul lui Eratostene	$O(n \log n)$	$O(n)$
Ciur pentru τ , σ	$O(n \log n)$	$O(n)$

1. **Prime and Divisor Count Queries** (AlgoCode #4229)
2. **Prime Factorization Exponents** (AlgoCode #737)
3. **Prime Factor with Largest Exponent** (AlgoCode #727)
4. **Divisor Summary** (AlgoCode #4201)

1. **infoas.ro** — complexitate timp și spațiu în C++
2. **bigocheetsheet.com** — tabel vizual cu complexitățile comune
3. **pbinfo.ro** — Standard Template Library
4. **cppreference.com** — documentație completă C++